FalconVQA

Fast Augmented Language-based CONversational
Video Question Answering

A Semantic Retrieval Layer for Surveillance Footage

Software Design Document

Avinash Anish Vaivaswat Verma

August 6, 2026

Contents

1 Introduction and Problem Understanding

1.1 System Identification

This document specifies the design of **FalconVQA** — *Fast Augmented Language-based CONversational Video Question Answering* — a retrieval and question answering system for recorded video, developed with surveillance footage as its primary application domain. The name records the four properties the system is designed around: retrieval that is *fast* enough to be interactive, an index *augmented* beyond the raw signal with structured descriptions produced by vision and audio models, a *language-based* interface in which both the index and the query are natural language, and a *conversational* access path in which an operator or an autonomous agent asks questions of a recording and receives cited answers rather than a list of files.

1.2 Problem Statement

Recorded surveillance exists as a continuous timeline indexed by nothing but a clock. Locating an incident therefore requires the operator to select a date and time and review the footage manually until the incident is observed. This procedure is tractable only when the time of the incident is already known. In practice it is not: what is available is a description of the event, obtained from a witness or a report — for example, *a man in a yellow shirt left a bag near the checkout*. No stage of the recording chain produces a record of what the footage contains, so no such description can be used as a query.

General-purpose video understanding tools do not close this gap, because they are designed for edited media and depend on three properties that surveillance footage does not exhibit:

- **Shot boundaries.** A fixed camera produces none.
- **Speech.** A substantial proportion of surveillance footage is silent.
- **Visually salient events.** Events of interest are typically brief and visually unremarkable.

Applied to such footage, these tools either return the entire recording as a single segment or subdivide it into a large number of near-identical segments. Neither result is searchable.

The problem addressed by this project is therefore twofold: to derive a descriptive representation of footage that possesses little intrinsic structure, and to make that footage retrievable through that representation. The operator states what is being looked for in natural language and receives the corresponding moment, timestamped and ready for playback.

1.3 Project Objectives

The system is required to deliver the following:

O-1. A processing pipeline that accepts a recording and renders it searchable by description.

O-2. Semantic search returning ranked moments with timecodes, and question answering that returns a synthesised answer with references to the moments it was derived from.

O-3. Video-level output that can be read in place of watching the footage: a summary, chapters, notable events, the people and objects that appear, and any anomalies.

O-4. A multi-tenant web application in which uploading, searching, questioning and playback are performed in a single workspace, with each operator's recordings isolated from every other operator's.

O-5. A documented and self-describing HTTP API, so that the same capabilities can be driven by an LLM agent rather than only by a human operator.

1.4 Document Organisation

Section 2 presents the solution and the processing pipeline in outline. Section 3 states the functional and non-functional requirements. Section 4 records the design enhancements that distinguish the system from a conventional caption-and-embed pipeline. Section 5 is the detailed design: each of the four pipeline stages, the models employed, the parameters governing them and the outputs produced. Section 6 specifies the architecture and technology stack, and Section 7 the database design across all four stores. Section 8 describes the user interface and the acceptance test plan. Section 9 sets out the execution schedule and the distribution of work.

2 Solution Overview

FalconVQA processes a recording in five stages: the video is partitioned into chunks, each chunk is analysed, the resulting records are indexed, video-level aggregates are derived from those records, and the indexed material is then queried by search or by question. Most stages are parameterised rather than fixed, because surveillance footage varies too widely for a single configuration to be appropriate in all cases. This section states what each stage does; Section 5 specifies the models, parameters and outputs of each in full.

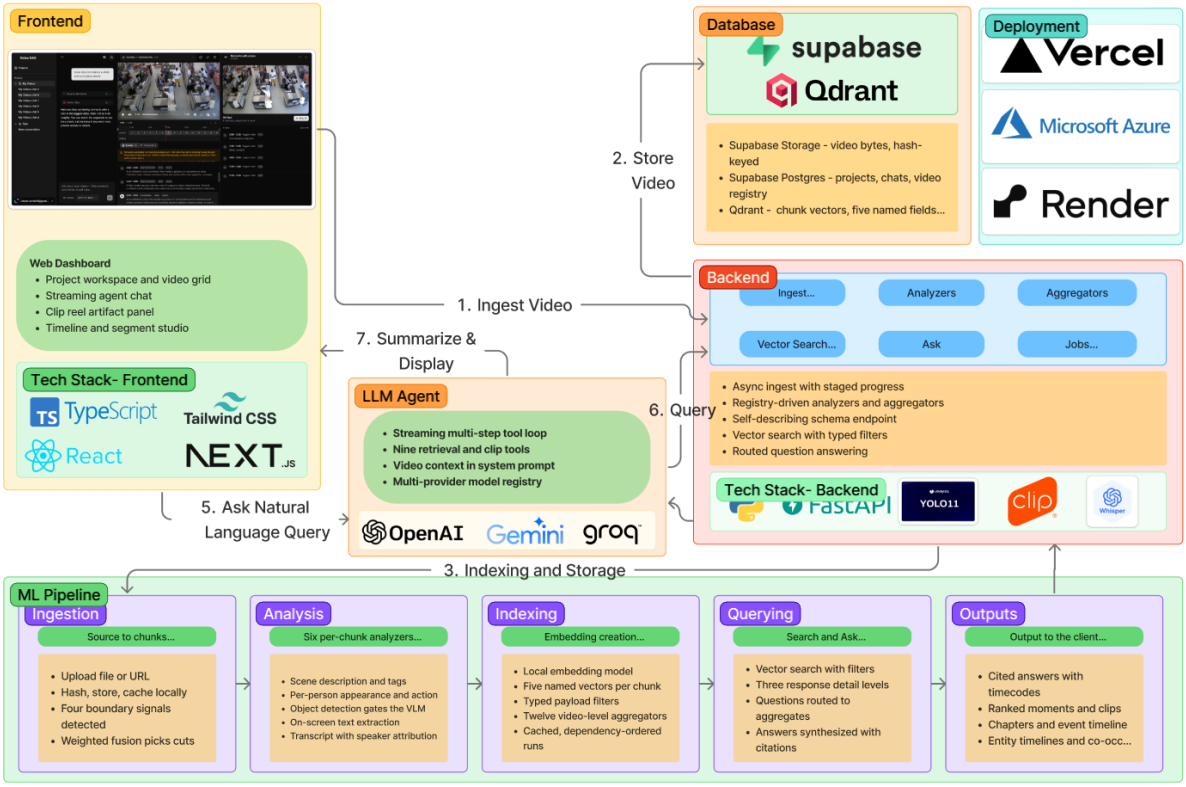

Figure 1: FalconVQA system architecture: client and agent tier, core analysis service, persistence, deployment targets, and the five-phase processing pipeline.

2.1 Partitioning the Recording

The recording is cut where its content changes. Four boundary detectors contribute evidence for a cut: a change of speaker, a period of silence, a hard scene cut, and a shift in the visual content of the frame. Their scores are fused under a weighting, and the strongest peaks of the fused signal are taken as boundaries.

Where a signal produces no events at all, its weight is redistributed across the signals that did fire. This behaviour is essential rather than incidental: fixed-camera footage contains no hard cuts and frequently no speech, and without redistribution the entire recording is returned as a single chunk. Two further modes are provided for cases in which fusion is not wanted — a caller-supplied weighting, or fixed spans of N seconds with no detectors run at all.

The chunking configuration is stored alongside the resulting records, so that a single recording may be indexed both coarsely and finely at the same time, with a query resolving against whichever indexing is appropriate.

2.2 Analysing Each Chunk

The analyzers to be run are selected at upload time, because their costs differ by orders of magnitude. Six analyzers are provided: scene description, per-person description, object detection, on-screen text recognition, transcription, and speaker-attributed transcription. Transcription and speaker-attributed transcription are mutually exclusive, as the latter is a strict superset of the former.

The vision–language model is the dominant cost, so inexpensive detectors are run first and act as gates. If YOLO detects no object in a frame and EasyOCR finds no text, that frame is never submitted to the vision model. Frames are additionally deduplicated: a frame is retained only if it differs sufficiently from the last frame retained. The similarity threshold

is adaptive with an absolute ceiling, a configuration arrived at by measurement — a fixed threshold retained a single frame on a static camera, and a purely relative threshold began selecting compression noise. Frames are read sequentially rather than by seeking, which was measured to be approximately 22× faster.

2.3 Indexing and Aggregation

Each chunk is stored as a single point carrying five separately embedded vectors rather than one: the complete flattened record, the scene description, the people, the actions, and the objects. A query concerning a person's appearance is therefore matched against a short description of a person rather than competing against everything else recorded for that chunk. Embeddings are computed locally, and each point carries a typed payload against which structured filters are applied.

Twelve aggregators then run over the saved analyzer output — not over the video — in an order derived from their declared dependencies. They produce the video-level material: statistics, novelty, speaker statistics, summary, chapters, events, named entities, sentiment, linked person entities, linked object entities, entity timelines and co-occurrence. Because they consume stored records, re-running an aggregator incurs no re-analysis; results are cached and recomputed only on demand or when the analyzer set changes.

2.4 Search and Question Answering

Searches may be restricted by time, objects, tags, people and other typed filters, and return one of three levels of response detail, selected according to whether the consumer is a human reader or a token-metered agent.

Questions follow a different path. Rather than searching chunks and synthesising from whatever is returned, the question is routed to the video-level aggregates capable of answering it: the entity narratives for a question about a person, novelty for a question about anomalies, statistics for a question about activity levels.

In both cases the operator supplies a natural-language description and receives ranked moments carrying `mm:ss` timecodes. Selecting a result begins playback of the recording at that point.

3 Requirements Analysis

3.1 Functional Requirements

Functional requirements are stated from the operator's perspective: what a user of the system must be able to do with their footage. Requirements FR-1 to FR-12 are implemented across the client application and the core analysis service.

ID	Capability	The system shall allow the operator to...
FR-1	Account and workspace	Register, authenticate, and organise recordings into named projects, with each operator able to reach only their own projects and recordings.
FR-2	Adding footage	Add a recording to a project, either as an uploaded file or as a link, and select how thoroughly it is to be examined — trading processing cost against the depth of detail captured.
FR-3	Tracking progress	Observe that a recording is being processed, which stage it has reached and how far it has advanced, and continue working while processing proceeds.
FR-4	Searching	Locate moments by describing what occurred in natural language, without knowing when it occurred or what it was called.
FR-5	Narrowing results	Restrict a search to a period of the recording, to footage containing particular people, objects or spoken content, or to a chosen subset of recordings.
FR-6	Asking questions	Pose a question about one or more recordings and receive a direct answer accompanied by references to the moments from which it was derived.
FR-7	Reviewing results	Determine exactly when each result occurred and play the footage from that moment, individually or as a continuous reel of moments.
FR-8	Understanding a recording	Review a recording as a whole — its summary, chapters, notable events, the people and objects that appear and when, and any anomalies — without watching the footage.
FR-9	Conversing about footage	Conduct a multi-turn conversation about a project's recordings in which the assistant selects the appropriate retrieval operations, and have that conversation persist for later resumption.
FR-10	Inspecting the index	Examine what the system recorded for any individual chunk — scene description, on-screen text, people, objects and speech — alongside a timeline synchronised with playback.
FR-11	Re-examining footage	Re-examine a recording already held by the system at a different depth of analysis, without re-adding it and without discarding results already obtained.
FR-12	Programmatic access	Drive every retrieval capability over a documented HTTP API whose analyzers, filters and response shapes are discoverable from the API itself.

3.2 Non-Functional Requirements

Non-functional requirements are stated as service targets an operator would observe. The stated figures are design budgets to be confirmed by benchmarking, not measured results.

ID	Quality	Requirement
NFR-1	Responsiveness	A description search returns ranked moments within 2 seconds; a question is answered within 15 seconds.
NFR-2	Processing time	A recording becomes searchable within approximately three times its own duration, and the operator may continue working on other recordings while it is processed.
NFR-3	Transparency	Processing stage and progress remain visible throughout ingestion, refreshed at least every 5 seconds, so that a prolonged wait is never unexplained.
NFR-4	Playback latency	Selecting a result begins playback of the footage at that moment within 2 seconds, with no manual scrubbing.
NFR-5	Traceability	Every answer cites the moments from which it was derived, and the system states that the footage does not support an answer rather than producing one.
NFR-6	Robustness	Ordinary surveillance footage — unedited, frequently silent, recorded from a fixed camera — is handled as the normal case rather than as a failure case.
NFR-7	Cost control	The operator selects how thoroughly each recording is examined, and reviewing results already obtained incurs no further processing cost.
NFR-8	Extensibility	A new analyzer or aggregator is added as a single module and one registry entry, with no modification to ingestion, storage or the API.
NFR-9	Data isolation	Recordings, projects and conversations are readable only by the account that owns them, enforced at the database and at the boundary through which the agent reaches the analysis service.
NFR-10	Idempotent ingestion	A recording is identified by the content of its bytes, so that re-submitting the same file is recognised as the same recording and is neither stored nor processed twice.

4 Creative Enhancements

The enhancements recorded below were, in most cases, adopted in response to a measured failure on real footage. Collectively they are what distinguishes the system from a conventional "partition the video and embed the captions" pipeline. Each is annotated with the requirement it primarily serves.

#	Enhancement	Rationale	Serves
E-1	Multi-signal chunking with weight redistribution	Four detectors contribute evidence for each cut, and a signal that produces no events cedes its weight to the signals that did. Without this, fixed-camera footage is returned as a single chunk, and a preset degenerates into a granularity dial: on single-signal footage the `audio` and `video` presets yielded 12 and 63 boundaries respectively from identical content.	NFR-6
E-2	Concurrent chunkings of one recording	The chunk configuration is stored with the records, so a recording may be indexed coarsely and finely simultaneously and a query resolves against whichever is appropriate.	FR-11
E-3	Five named vectors per chunk	Description, people, actions and objects are embedded separately alongside the combined record, so a query about a person's appearance is matched against a short person description rather than competing with everything else recorded for that chunk.	FR-4
E-4	Inexpensive detectors used as gates	YOLO and EasyOCR determine whether a frame merits submission to the vision–language model. Frames containing nothing to see or read incur no model cost.	NFR-7
E-5	Detector labels withheld from the vision model	YOLO classified a thermally imaged tank as an "airplane" with 0.73 confidence. Shown the same bounding boxes without labels, the vision model reported a tank. A label rendered onto the image invites agreement with it.	NFR-5
E-6	Adaptive frame sampling	Frames are retained according to their dissimilarity from the last frame retained. A fixed threshold of 0.95 retained one frame on a static camera, whose consecutive similarity median was 0.9955; a purely relative threshold selected compression noise. Both were measured.	NFR-2
E-7	Questions routed to aggregates	A question is directed to the video-level output capable of answering it — entities for a person, novelty for anomalies, statistics for activity levels — rather than searching segments and synthesising from whatever is returned.	FR-6
E-8	Constrained entity linking	People are clustered on appearance under hard constraints — two people visible in the same chunk cannot be the same person — *before* an LLM writes any narrative. Asked to match people directly, an LLM merges anyone in dark clothing. The signature is clothing alone: blending in the appearance field dragged same-person scores to 0.69–0.87, because it drifts and occasionally carries meta-commentary that the embedding treats as content.	FR-8
E-9	Object linking with fixture suppression	Object entities reuse the same constrained clusterer with two additional filters: static fixtures such as counters and shelves are counted but not tracked, since "the counter was present throughout" is true of every frame and informs no search; and person-like detections are handed to the people aggregator, which has clothing with which to identify them.	FR-8
E-10	Selectable response detail	The same search returns a few hundred tokens or tens of thousands. Five full hits were measured at approximately 19,000 tokens against approximately 440 at `minimal`. A human reader wants the whole record; an agent pays per token.	NFR-7
E-11	Self-describing API	A single endpoint returns every analyzer, vector field, filter and aggregator the installation supports, so a client or an LLM agent discovers what may be asked instead of being hard-coded against it. The upload dialog builds its analyzer list from this endpoint, so that adding an analyzer remains a change to one module and one registry line.	FR-12, NFR-8
E-12	Content-addressed recordings	A recording is identified by the hash of its bytes, so the same file submitted twice is recognised as the same recording and is neither processed nor stored again. No local path is ever persisted; the cached file is re-derived from the content hash and re-downloaded if the cache is cold.	NFR-10

5 Detailed Design of the Processing Pipeline

This section specifies each of the four pipeline stages in turn: the models employed, the parameters governing them, and the output each stage produces for the next. Every numeric value given is the value in the implementation.

5.1 Stage 1 — Chunking

Purpose and inputs

The stage receives a decoded video file and produces an ordered list of non-overlapping time spans covering the recording. Audio is decoded once with PyAV into a mono waveform, from which the duration is derived; the video stream is read separately by the visual detectors.

Boundary detectors

Four detectors run once per recording. Each returns a list of (time, strength) events, where the strength lies in $[0, 1]$.

Signal	Model / algorithm	Event derivation	Strength
Speaker change	`pyannote/speaker-diarization-community-1` (gated, requires a Hugging Face token)	The waveform is diarized into speaker turns; a boundary is emitted at the midpoint between two consecutive turns whose speaker label differs.	Constant 1.0. A detected speaker change is a binary, high-confidence event.
Silence	Silero VAD v6	Speech timestamps are returned for the whole track; a boundary is emitted at the midpoint of every gap between speech segments longer than 0.05 s, including the trailing gap.	$\min(1, g/2.0)$ for a gap of g seconds, saturating at two seconds.
Hard cut	PySceneDetect `ContentDetector`; purely algorithmic frame differencing, no model weights	A boundary is emitted at the start of every scene after the first.	Constant 1.0. The detector applies its own threshold, so any report is confident.
Semantic drift	OpenCLIP ViT-B/32, OpenAI weights, frames sampled at 1 fps	Consecutive sampled frames are embedded and compared; a boundary is emitted at each midpoint with raw strength $1 - \cos(e_i, e_{i+1})$.	Rescaled by the clip's own largest jump, so the strongest semantic change is placed on equal footing with a speaker change or a cut.

The rescaling applied to the semantic signal is necessary because CLIP similarity drifts continuously and rarely approaches zero even at a genuine scene change; without it the signal would be permanently diluted relative to the sparse, unit-strength detectors.

Frames for the semantic detector are read by walking the container forward with `grab()`/`retrieve()` rather than seeking to each timestamp. Per-frame seeking forces the H.264 decoder to restart from a keyframe and was measured at approximately 22× the cost per frame.

Fusion

Weights are first renormalised over the signals that produced at least one event, so that a preset expresses *which signals matter* rather than acting as a granularity dial (Section 4, E-1). Each event then contributes a Gaussian bump to a score curve evaluated on a grid of resolution 0.1 s over the recording:

$$S(t) \;=\; \sum_{s \in S} \; \sum_{(t_i,\, a_i) \in E_s} w_s \, a_i \, \exp\!\left(-\frac{(t - t_i)^2}{2\sigma^2} \right), \qquad \sigma = 1.0 \text{ s}$$

Here S is the set of signals, E_s the events emitted by signal s, w_s its renormalised weight, a_i the strength of event i, and σ the width of the Gaussian kernel.

Boundaries agreed upon by several detectors therefore reinforce one another. Peaks are selected greedily in descending order of $S(t)$, subject to a score floor of 0.12 and a minimum separation of 2.0 s between accepted boundaries, which prevents runs of adjacent micro-chunks.

Preset	speaker	silence	cut	semantic	Intended footage
`audio`	0.35	0.35	0.15	0.15	Dialogue-led recordings
`video`	0.15	0.15	0.35	0.35	Edited or visually varied footage
`audio_video`	0.25	0.25	0.25	0.25	Default; all four signals equally

Modes and post-processing

Three mutually exclusive modes are exposed. In `preset` mode a named weighting is used; in `weights` mode the caller supplies its own, validated against the four signal names and required to be non-zero in at least one; in `interval` mode the recording is cut into fixed spans of N seconds and no detector runs at all.

For the two weighted modes, a span of length d longer than the maximum duration $D_{\max}$ is divided into $\lceil d / D_{\max} \rceil$ equal parts — equal division rather than truncation, so that no negligible trailing fragment is produced — and a span shorter than the minimum duration is merged forward into its successor, with a short final span folded back into the preceding one. The API defaults are 5 s and 20 s. Duration bounds are deliberately *not* applied in `interval` mode, since the caller requested exact spans.

Output

An ordered list of (start, end) spans, together with a chunk configuration key that identifies the scheme that produced them — for example `audio_video:5-20` or `interval:10`. Custom weightings are hashed into the key so that two different weightings sharing the same duration bounds cannot collide. This key is carried on every downstream vector, which is what permits two chunkings of one recording to coexist.

5.2 Stage 2 — Analysis

Purpose and inputs

Each selected analyzer receives the chunk spans and a video context that provides frames and audio on demand. Every analyzer owns its own frame sampling, its own prompt and its own output schema, and is registered by identifier; nothing in ingestion, storage or the API names a specific analyzer.

Common sampling strategy

The frame-selecting analyzers share a three-part strategy. Candidate frames are drawn at a low fixed rate; near-duplicates are removed by comparing OpenCLIP embeddings against the last frame retained; and the survivors are thinned to a per-analyzer maximum. The scene analyzer derives its similarity threshold adaptively, as the 40th percentile of consecutive-frame similarity within the chunk, capped at 0.98. The cap is load-bearing: on a static chunk the percentile lands on the median, measured at 0.9955, and distinct frames are then mined out of compression noise. The detector-gated analyzers instead use a fixed similarity threshold together with a layout-change test, because a person or object entering or leaving the frame is a change the whole-image comparison does not register.

All frames selected for a chunk are sent in a single request, so that the model sees the chunk as a continuous sequence and can describe movement across it rather than judging each frame in isolation. Sampling is serial, because the CLIP model is shared on one GPU; the API calls are I/O-bound and run across six worker threads.

Analyzers, models and outputs

The vision–language model is invoked with OpenAI strict structured output, so every required property is present and no additional properties are returned; downstream stages therefore never parse prose.

Two design rules apply across the table. First, whole-track passes are performed once and memoised: a per-chunk diarization pass would label the same person `SPEAKER_00` in one chunk and `SPEAKER_01` in the next, and per-segment language detection produced fluent German from an English clip. Second, `transcript` and `diarization` are mutually exclusive and the API enforces this, because their embedded vectors were measured cosine-identical at 1.0000 — diarization is a strict superset.

Gating and cost

Where an analyzer's gate finds nothing — no person above the area threshold, no object detection, no text region — the chunk yields no output and no API call is made at all. A frame that survives is downscaled to the analyzer's width and JPEG-encoded before transmission, since vision tokens are priced by resolution.

Analyzer	Models and gating	Structured output
`default_video`	Candidates at 2 fps; OpenCLIP ViT-B/32 deduplication (adaptive threshold, cap 0.98); 4–8 frames retained, thinned against evenly spaced points in *time*; VLM `gpt-5.4-mini` at 768 px.	Schema `scene_description`: `description` (the prose users search against), `setting`, `people[]`, `objects[]`, `actions[]`, `tags[]`. The timestamps of the frames used are recorded alongside for traceability.
`people`	YOLO11n gate at 1 fps, confidence 0.35; person boxes below 900 px^2 discarded; identifiers assigned by overlap tracking across *all* candidate frames, then read off for the frames actually sent; up to 6 frames annotated with numbered yellow boxes; VLM at 1280 px, `detail=high`.	Schema `people_reading`: per person `box_id`, `appearance`, `clothing`, `role`, `action`, plus a `summary`. Each person is post-processed with `locations` — the timestamp and box in every frame they appear in — and a `people_count` is recorded.
`object_ detection`	Shares the same YOLO11n pass rather than repeating it; frames annotated with cyan *unlabelled* boxes; up to 6 frames; VLM at 1280 px, `detail=high`.	Schema `object_reading`: `detections[]` of `object`, `description` (colour, material, size, condition) and `context` (what it is being used for, and by whom), plus a `summary`. Plain object names are extracted into a filterable list; the detector's own labels are retained only for debugging.
`ocr`	EasyOCR *detection* stage only — its recognition stage is skipped, as it fragments lines and reads low-contrast overlays poorly; frames annotated with red boxes; similarity threshold 0.985 plus a text-layout change test; VLM at 1600 px, `detail=high`.	Schema `ocr_reading`: `texts[]` of `text` (verbatim, with case and punctuation) and `context` (what the text is and where it appears), plus a `summary`.
`transcript`	faster-whisper (`tiny`); language detected once over the whole track and pinned for every chunk.	One plain-text transcription per chunk.
`diarization`	pyannote over the whole track once, memoised; faster-whisper segments per chunk; speaker assigned by largest temporal overlap between a Whisper segment and a pyannote turn.	`turns[]` of `speaker`, `start`, `end`, `text` with consecutive same-speaker segments merged, plus a rendered `text`, a speaker-free `plain` form, and the `speakers` present.

5.3 Stage 3 — Indexing and Aggregation

Embedding

Every analyzer implements `render_fields`, which flattens its output into a map of named vector space to text. Five spaces are defined — `combined`, `description`, `people`, `actions`, `objects` — of which `combined` must always be present. A space is *omitted* rather than emitted empty, so a chunk in which the model saw nobody is simply not a candidate in a people-scoped search.

Text is embedded locally with `BAAI/bge-small-en-v1.5` through sentence-transformers, producing normalised vectors compared by cosine distance. The model family was trained with an instruction prefix on the query side only, so queries are prefixed with *"Represent this sentence for searching relevant passages:"* and documents are not; applying the prefix to both, or to neither, measurably degrades recall. All fields of all chunks are embedded in one batched GPU pass and then scattered back to their (chunk, field) slots.

Vector store

One Qdrant point is written per chunk per analyzer, carrying all five named vectors. Because the point is the unit, a limit of ten returns ten chunks, the payload is stored once, and a filter is applied once. The point identifier is a deterministic UUID derived from `sha1(video_id | analyzer_id | start | en` keyed on the time span rather than a positional index — chunk 12 of one chunking run is a different moment from chunk 12 of another, so a positional identifier would silently rebind a vector to the wrong timeframe. Re-ingesting the same chunk therefore upserts rather than duplicating.

The payload carries everything required to cite and render a moment without joining back to disk: the video identifier and playback URL, the analyzer, the chunk configuration and identifier, the time span, the combined text and description, the setting, the people, objects, actions and tags, the speakers and their turns, the read texts, the object detections, the full per-person records, and the person count. Twelve payload fields are declared as indexes, and twelve named filters are defined in a single filter specification — video, chunk and analyzer identifiers, chunk configuration, objects, tags, speakers, people, minimum and maximum person count, and time bounds. Filters are validated against that specification and an unknown name raises an error with the nearest suggestion, because a silently ignored filter returns plausible but wrong results.

Aggregators

Twelve aggregators run over the saved analyzer records, in an order derived from their declared dependencies. A dependency naming an analyzer is a requirement on the recording, and an aggregator whose analyzer is absent is dropped rather than run against nothing. Five of the twelve incur API calls; the remainder are free, which is what makes re-running them on demand practical.

Entity linking in detail

Identity is decided by embedding and clustering, not by asking a model to match people. Measured on this footage, provably different people — co-visible within one chunk, so they cannot be the same — reached 0.941 similarity, while known-same pairs sat between 0.889 and 0.915. Those bands overlap, so no threshold alone is sufficient and the constraints carry the weight. The procedure is:

1. **Signature construction.** The signature is the `clothing` field alone. Blending in `appearance` dragged same-person scores down to 0.69–0.87, because that field drifts between chunks and

Aggregator	Depends on	Model	Output
`stats`	—	None; arithmetic	Duration, chunk count, person count series with minimum, maximum, mean, busiest and quietest spans; object frequencies; word count, spoken seconds and speech ratio.
`novelty`	—	None; reuses stored vectors	Cosine distance of each chunk's `combined` vector from the normalised centroid of the recording, ranked; outliers are those beyond two standard deviations, so "unusual" scales with how varied the recording itself is.
`speaker_stats`	`diarization`	None; arithmetic	Per speaker: seconds, turns, words, share of speech, longest turn and chunks. Plus handover count, dominant speaker and a turn timeline.
`summary`	—	LLM	A hierarchy built by repeated halving. Depth is derived from chunk count — enough halvings for a leaf to cover at most five chunks, bounded to between three and six tiers — so that leaf granularity does not vary with recording length. Leaves are summarised from chunk text and every tier above merges its children, making the whole-recording summary a reduction rather than one enormous prompt. Emits `summary`, `key_points[]`, `topics[]`, the full tier hierarchy, and the finest tier as `sections` for retrieval.
`chapters`	—	LLM	Runs of consecutive chunks grouped into named sections with a title and one-sentence summary. Chapters must be consecutive, non-overlapping and exhaustive; any chapter referencing a chunk identifier that does not exist is discarded.
`events`	—	LLM	Discrete, chunk-anchored occurrences: `event`, `chunk_id`, `actor`, `category`, resolved to real time spans and counted by category. Ongoing background activity is explicitly excluded.
`ner`	—	GLiNER `urchade/gliner_small-v2.1`, zero-shot, threshold 0.5	Named entities over descriptions, on-screen text and speech across ten configurable labels. Pronouns and bare determiners are excluded by a stop list, as they are grammatically entities but identify nobody and would dominate a mention count. Emits entities with mention counts, scores and chunk identifiers, grouped by label.
`sentiment`	`diarization` (falls back to `transcript`)	`cardiffnlp/twitter-roberta-base-sentiment-latest`	A per-turn timeline, per-speaker distribution with dominant label and negative share, and overall distribution. Run on speech only, and reported as sentiment of language observed rather than emotion inferred.
`entities`	`people`	BGE embeddings and constrained clustering, then LLM narratives	Person observations linked into individuals. See below.
`object_entities`	`object_detection`	The same clusterer, plus lexical filters	Object sightings linked into individual objects. Static fixtures are counted but not tracked, and person-like detections are handed to the people aggregator.
`entity_timelines`	`entities`	None	Per person: first and last sighting, total span, *observed* presence summed over sight-

occasionally carries meta-commentary that the embedding treats as content.

2. **Within-chunk merge.** Observations in the same chunk above a similarity of 0.95 — above the measured co-visible ceiling — are merged first. The analyzer's own tracker loses a person who moves between sampled frames, and without this step the cannot-link constraint would permanently forbid reuniting them.

3. **Distinctness gate.** Each observation's mean similarity to all others is computed. Anything at or above 0.80 describes half the recording and identifies nobody, so its owner is left unlinked rather than merged into whichever cluster it drifted closest to.

4. **Constrained agglomeration.** Pairs above 0.88, drawn from different chunks and both distinctive, are merged greedily in descending order of similarity. A merge is rejected if the two clusters share any chunk, since that would place one person in two places at once.

5. **Narration.** Only then is an LLM used, and only for people seen more than once, capped at twelve narratives per recording. It produces a one-line recognition description, an ordered account of what the person did, and an apparent role.

Object linking reuses the same routine on a source where identity is inherently weaker — two shopping carts genuinely look alike where two people rarely wear the same clothes — so the distinctness gate does most of the work.

5.4 Stage 4 — Retrieval and Question Answering

Search

A search embeds the query with the query-side prefix, selects one of the five named vector spaces, applies the validated filter set, and returns ranked chunks with their time spans and payloads. Two parameters deserve note.

Score threshold. Cosine similarity always ranks something, so a query for content a recording does not contain still returns its nearest neighbours. A score threshold is what makes "no match" expressible. On this data, content actually present scored ≥ 0.665 and absent content ≤ 0.524, so a threshold of approximately 0.55–0.60 separates them; the value is specific to the embedding model and is retuned if the model changes.

Detail level. Three levels are offered — `minimal`, `standard` and `full`. Five full hits were measured at approximately 19,000 tokens against approximately 440 at `minimal`. The interface requests the whole record; an agent, which pays per token, requests less.

Question routing

A question is not answered by searching chunks and hoping. Each aggregate carries a hint — a short description, written in the vocabulary a question would use, of what that aggregate is good for. The hints are embedded and compared against the question, and the aggregates standing at least one standard deviation above the mean score for that question are selected, capped at three. If nothing stands out, the single best-matching aggregate is used, so a broad question is answered from the strongest match rather than from nothing.

Relative selection is used because absolute thresholds do not survive contact with real questions: the score distribution shifts per question, with means between 0.47 and 0.56 measured here, so any single cutoff either admits everything or nothing. The ranking, however, is reliable. Because routing is by embedded description rather than by keyword, a new aggregator joins by adding one line of hint text, and unanticipated phrasings degrade gracefully.

Aggregate	Questions it is routed
`entities`	What a particular person did; someone identified by clothing or appearance; following one individual; how long a person stayed.
`novelty`	Which segment is unusual, anomalous or out of the ordinary; what stands out.
`stats`	How many people were present; how busy or crowded; the busiest or quietest moment; counts and totals.
`events`	What happened and when; arrivals, departures and transactions in time order.
`chapters`	The structure of the footage; what sections it has; what the recording is about overall.
`speaker_stats`	Who talked most; how many speakers; speaking time, turns and interruptions.
`sentiment`	The tone of what was said aloud; whether spoken language was positive, negative or neutral.
`ner`	Named things mentioned: brands, organisations, places, names, dates and amounts.
`object_entities`	A particular object followed through the footage; what an item was used for over time.
`cooccurrence`	Which people appeared together; who was accompanied by whom.

Context assembly and synthesis

The routed aggregates are rendered into labelled blocks, each carrying explicit timecodes. The overall summary is always included, together with the four summary sections closest to the question and, where entities were routed, the four people whose descriptions best match it. Matching chunks from vector search are appended last. The response records which aggregates were routed to and how much of each was used, so an answer can be traced to its sources.

The synthesis prompt constrains the model to the supplied material, requires every claim to carry a `[video_id start-end]` citation drawn from the timecodes given, and directs it to state plainly that the material does not support an answer rather than guessing (NFR-5).

Output to the client

The stage returns cited answers with timecodes, ranked moments with playable time ranges and playback URLs, and — from the aggregates — chapters, the event timeline, entity timelines, co-occurrence, statistics, novelty and sentiment. A moment is delivered as a range over a recording rather than as a rendered file, so several moments drawn from one recording share a single media load in the client.

6 Architecture and Technology Stack

6.1 Design Methodology

The system was developed iteratively, one pipeline stage at a time, each stage operating end to end before the next was begun. Two principles governed most of the design decisions:

- **Measurement precedes commitment.** Every threshold and sampling choice in the pipeline was established by executing it against real footage and comparing outcomes, rather than by selecting a plausible value. Several of the enhancements listed in Section 4 exist because the obvious choice was adopted first and observed to fail.
- **Extension by registration, not by modification.** Analyzers and aggregators are declared in a single registry and discovered from it. No part of ingestion, storage or the API names a specific analyzer, so introducing one does not require any of them to be edited.

Verification was performed against a running server rather than by code inspection, because a number of defects — stale processes, broken answer synthesis, incorrect media paths — were observable only over HTTP.

6.2 Architectural Overview

The system is a three-tier application. A Next.js client provides the operator interface and hosts an LLM agent that translates conversational requests into retrieval calls. A FastAPI service owns the entire video pipeline and exposes it as a self-describing HTTP API. Persistence is divided across object storage for media, a relational database for application state, and a vector database for retrieval.

Composition within the analysis service is deliberately *in-process*: aggregators consume analyzer output directly rather than over HTTP, and HTTP is reserved for the external boundary. Analyzers and aggregators are held in registries, which is what makes NFR-8 achievable — the ingest path, the store and the API iterate over a registry and never name a specific analyzer.

The analysis service has no concept of users, projects or row-level security: any caller able to reach it can read every recording it holds. Multi-tenancy is therefore enforced entirely in the client tier, by row-level security on the database and by a single scoping function through which every agent tool resolves the recording identifiers it is permitted to pass downward (NFR-9).

Layer	Responsibility
Client	Marketing site and documentation, authentication, project workspace, video upload and status, agent chat, clip artifact panel, video studio timeline, and the per-recording detail view.
Agent layer	Multi-step streaming tool loop. Converts a natural-language request into search, question, inspection and clip operations, and streams results and artifacts back to the client.
API route layer	The client's own server routes: project scoping, row-level security, job reconciliation, and proxying to the analysis service.
Core service	Ingest orchestration, chunking, analyzers, aggregators, vector search, question answering, clip and poster export, job tracking.
Storage boundary	The single module that knows both a storage URL and a local path; everything above it speaks URLs, everything below it receives paths.
Persistence	Object storage for video bytes, relational store for application state, vector store for chunk embeddings, local cache for decoded media and model weights.

6.3 Module Inventory

Tier	Module	Responsibility
Core	`chunk.py`, `chunking/`	The three chunking modes and the weighted fusion of boundary evidence.
Core	`boundaries/`	The four detectors: speaker change, silence, hard cut, semantic drift.
Core	`analyzers/`	Six per-chunk passes, each owning its frame sampling, prompt and output shape.
Core	`aggregators/`	Twelve video-level passes over saved records, ordered by declared dependencies.
Core	`extractors/`	Shared primitives: frame readers and samplers, audio decoding, vision–language calls, detector wrappers.
Core	`vectordb/`	Local embedding, the Qdrant point schema with five named vectors, and record rendering.
Core	`api/`	`core.py` holds the logic; `app.py` is a thin HTTP layer; `ask.py` performs routed question answering; `jobs.py` tracks background ingestion.
Core	`storage.py`, `paths.py`	The storage boundary and every filesystem path, each overridable by environment variable.
Core	`export.py`, `poster.py`	Per-chunk MP4 export by a single decode pass, and one poster frame per recording.
Client	`app/projects/`	Project list, workspace, upload dialog, media grid, and the per-recording detail view.
Client	`app/agent/`	Chat surface, message and tool renderers, artifact panel, clip reel, and the video studio timeline.
Client	`app/api/agent/tools/`	The nine agent tools and the project scoping function that binds them.
Client	`app/api/videos/`	Ingestion, status, aggregates, timeline, details and re-index routes.
Client	`lib/core/`	Typed client for the analysis service, job reconciliation, chunking configuration and formatting helpers.
Client	`lib/supabase/`	Browser, server and admin clients, session middleware, and the schema migrations.
Client	`content/docs/`	The MDX documentation site: concepts, workspace guides and the full API reference.

6.4 Ingestion Lifecycle

Ingestion is asynchronous. The service accepts a file or a link, returns immediately with a job identifier, and processes the recording on a background thread through the stages specified in Section 5. The job reports its stage — fetching, chunking, analyzing, indexing, aggregating, complete — and the client polls it, reconciling the result into the recording's row. Because jobs are held in memory, a restart mid-ingest is detected by the client, which either recovers the recording from its persisted records or fails the row explicitly so that re-indexing becomes the route forward.

6.5 Persistence

State is divided across four stores: a relational database for application state, object storage for media, a document store on disk for analyzer and aggregator output, and a vector store for

retrieval. The division, the schema of each store and the identity that joins them are specified in Section 7.

6.6 Technology Stack

Concern	Technology
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4, shadcn/ui on Radix primitives
Agent & models	Vercel AI SDK v6; OpenAI, Google, Groq, Cerebras and xAI providers behind one model registry
Client state	Zustand, TanStack Query
Playback	video.js, hls.js
Documentation	Fumadocs over MDX, served from the same application
Auth & tenancy	Supabase Auth with Postgres row-level security
Backend	Python 3.13, FastAPI, Uvicorn
ML runtime	PyTorch 2.11 (CUDA 13), Transformers
Boundary detection	pyannote.audio (speaker change), Silero VAD (silence), PySceneDetect (hard cuts), OpenCLIP ViT-B/32 (semantic drift)
Analysis models	Vision–language model for scene, people, object and text description; faster-whisper for transcription; YOLO11 and EasyOCR as detection gates
Aggregation models	GLiNER for zero-shot named-entity recognition, a transformer classifier for sentiment, an LLM for narrative aggregates
Embeddings	BGE-small-en-v1.5 via sentence-transformers, executed locally
Vector store	Qdrant, one point per chunk with five named vectors
Media I/O	OpenCV, PyAV, soundfile, Pillow
Application data	Supabase — Postgres, Auth and Storage
Deployment	Vercel (client), CUDA-capable host (core service), Supabase Cloud, Qdrant

Justification of Principal Choices

- **Python for the analysis service.** Every model the system depends upon — diarization, CLIP, Whisper, YOLO, the embedding model — is distributed as a Python library first. No other runtime would have permitted this number of models to be assembled in the time available.
- **FastAPI.** Asynchronous request handling for long-running ingest jobs, and generated API documentation, which is material when an LLM agent is one of the clients.
- **Qdrant.** Named vectors are the determining factor. Storing several vectors per chunk under one point is a first-class capability there, and it is what enhancement E-3 rests upon. It also runs embedded, so development requires no separate server.
- **A local embedding model.** Embedding is performed on every chunk of every recording. Paying an API per embedding would make re-indexing expensive; a small local model makes it free.
- **Next.js and the Vercel AI SDK.** Streaming responses, tool calling and multi-provider model selection are provided, so the agent interface reduced to defining tools rather than building a chat runtime.

Figure 2: Technology stack across the frontend, backend, storage and deployment tiers.

- **Supabase.** Postgres, authentication, row-level security and file storage from a single service, which together constitute the whole of the non-video state.

6.7 System Requirements

Component	Requirement
GPU	CUDA-capable NVIDIA GPU, 8 GB VRAM minimum (developed on an RTX 4060). Required for diarization, CLIP, Whisper, YOLO and the embedding model.
Runtime	Python 3.13 for the analysis service; Node.js 20 or later for the client.
Memory / storage	16 GB system RAM; disk sized for the media cache, which holds one decoded copy of each ingested recording keyed by content hash.
External services	Object storage and Postgres (Supabase project); an LLM provider API key for vision analysis and answer synthesis; a Hugging Face token for the gated diarization model.
Network	Outbound HTTPS for model downloads, object storage and LLM calls. Ingestion executes as a background job, so link latency does not block the client.

6.8 Known Limitations

The following limitations are known and accepted for the current phase:

- **Entity linking is confined to a single recording.** The clustering step generalises to a camera network unchanged, but the available footage shares no people, so it could not be validated.
- **Within-chunk person tracking fragments.** Bounding-box association at 1 fps loses a person who moves between sampled frames, so a box identifier is a referent within a chunk rather than an identity claim.
- **Payload indexes are inert in embedded mode.** Filtering is correct but scans. Running Qdrant as a server resolves this with no code change.
- **Jobs are held in memory.** A restart discards job history; records and vectors survive on disk, and the client detects and recovers from the lost job.

- **Chapters may collapse to one** on unbroken single-location footage. This is arguably correct but of little use in the interface.

6.9 Scope for Future Enhancements

The design reserves room in the areas expected to be extended next:

- **Cross-recording person linking.** People are already linked within a recording; performing the same clustering step over a wider set extends this across an entire camera network once suitable footage is available.
- **Live camera input.** Ingestion presently accepts a recording. The same pipeline applies to a rolling window from a live feed, since chunking operates on segments rather than whole files.
- **Visual re-identification.** Where clothing descriptions are too generic to separate two people, the stored bounding boxes provide the crops required to compare them visually.
- **Additional analyzers.** Any pass that reads a chunk and returns structured output — licence plates, pose estimation, crowd density — may be added as a single module with no change elsewhere.
- **Agent access over MCP.** The API already describes itself; exposing it as a tool server would permit an external assistant to use it directly.

7 Database Design

7.1 Storage Strategy

Persistence is divided across four stores rather than one. The division is deliberate: the three kinds of state the system holds have incompatible access patterns, and forcing them into a single store would compromise all three.

Store	Technology	Contents	Unit
Application database	Supabase Postgres	Accounts, projects, the recording registry, conversations and their messages. Normalised, transactional, and subject to row-level security.	Row
Object storage	Supabase Storage, two buckets	Video bytes as uploaded, the canonical copy the pipeline reads, and one poster frame per recording.	Object
Analysis record store	JSON documents on local disk	Everything the analyzers and aggregators produced for one recording, nested under its chunks.	Document
Vector store	Qdrant, embedded	One point per chunk per analyzer, carrying five embeddings and a filterable payload.	Point

The analysis output is held as documents rather than as relations because its schema is defined by the analyzer registry, not by the database: each analyzer declares its own output shape, and a chunk carries one key per analyzer that ran on it. Modelling that relationally would make adding an analyzer a schema migration, which directly contradicts NFR-8. Conversely, application state is relational because it requires foreign keys, cascading deletes and per-row access control, none of which a document store provides as cheaply. Retrieval is separated again because approximate nearest-neighbour search over several named vector spaces is not something either of the other two can serve.

7.2 Application Database

Figure 3 gives the logical structure — which relations exist and how they are related — and Figure 4 the physical schema as it stands in Postgres, with every column, its type and whether it is nullable.

Entity relationships

Physical schema

`video_core`

This relation is the application's view of a recording analysed by the core service. It is the only place where the two halves of the system meet, and three of its design decisions are load-bearing.

D-1. Two URLs are kept, not one. The system spans two Supabase projects: this one holds the row and the object the browser uploaded, while the analysis service's own project holds the canonical copy that plays. `source_url` and `playback_url` therefore point at different objects and both are retained.

D-2. Uniqueness is per project, never global. Because `core_video_id` is the content hash, the same file added to two projects yields the same value. The unique index is declared over `(project_id, core_video_id)` and is partial — restricted to rows where

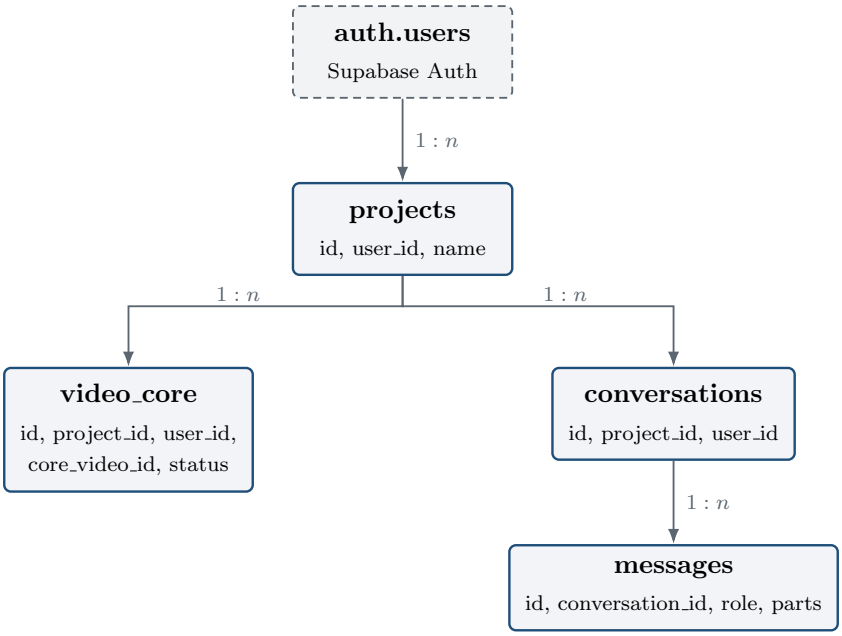

Figure 3: Application database. Every relationship cascades on delete, so removing a project removes its recordings, conversations and messages with it.

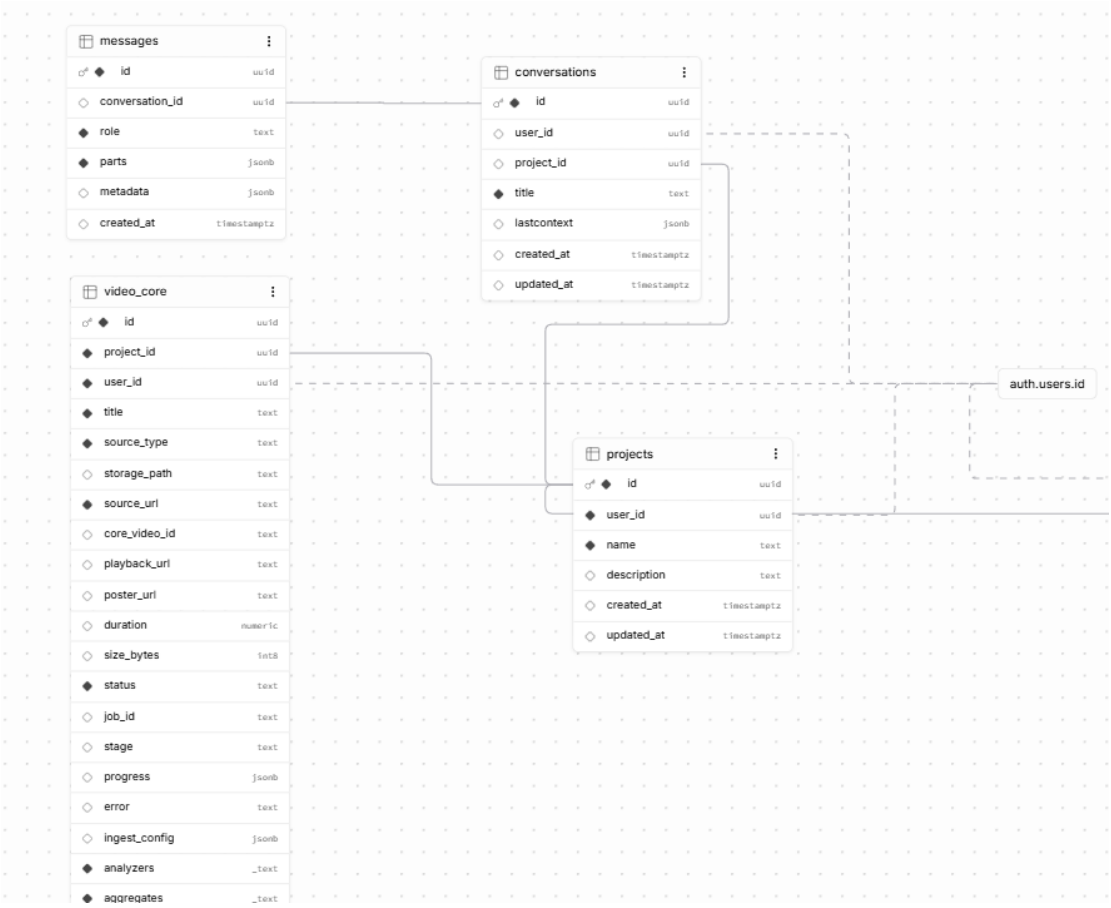

Figure 4: Application database as deployed: the four live relations with their columns, types and nullability, and their foreign keys into `auth.users`. Filled markers denote non-nullable columns.

Column	Type	Purpose
id	uuid PK	Application identity, independent of the analysis service.
project_id	uuid FK	Owning project; cascades on delete.
user_id	uuid FK	Owning account; the column row-level security tests.
title	text	Display name, editable without affecting analysis.
source_type	text	upload or url.
storage_path	text	Object in the application project's bucket.
source_url	text	The URL the analysis service was handed.
core_video_id	text	The content hash the analysis service identifies this recording by. Null until the bytes have been fetched.
playback_url	text	The public MP4 in the analysis project's bucket; this is what actually plays.
poster_url	text	Single frame written at ingest, for the workspace grid.
duration, size_bytes	numeric, bigint	Reported back by the analysis service.
status	text	pending, uploading, queued, analyzing, ready or failed.
job_id	text	The background job being polled.
stage	text	fetching, chunking, analyzing, indexing, aggregating or complete.
progress	jsonb	The job's last progress detail, rendered directly by the interface.
error	text	Failure reason, retained so the row explains itself.
ingest_config	jsonb	What the pipeline was asked for: analyzers, mode, preset, weights, interval and duration bounds.
analyzers	text[]	What it actually produced.
aggregates	text[]	Which video-level results exist.
chunk_config	text	The chunking scheme, e.g. audio_video:5-20.
chunk_count	int	Number of chunks produced.
created_at, updated_at	timestamptz	updated_at is maintained by trigger.

the column is non-null, since the identifier does not exist until the bytes have been downloaded. Deleting one project's row must therefore not delete the analysis service's copy while another row still references it.

D-3. Produced analyzers are an array, not a document. Both the interface and the agent evaluate *"does this recording have people analysis?"* as a predicate, which an array answers directly and an opaque JSON document does not. The requested configuration, by contrast, is `jsonb`, because it is replayed verbatim on re-index rather than queried.

Remaining relations

Relation	Columns and notes
`projects`	`id`, `user_id`, `name`, `description`, timestamps. The grouping every other relation hangs from.
`conversations`	`id`, `user_id`, `project_id`, `title`, `lastContext`, timestamps. A conversation is optionally attached to a project; the attachment is what scopes the agent's tools.
`messages`	`id`, `conversation_id`, `role`, `parts`, `metadata`, `created_at`. `parts` is `jsonb` because a turn is a sequence of heterogeneous parts — text, tool calls, tool results and artifacts — whose shapes are set by the agent runtime.
`videos`	Superseded. The relation predates the current analysis service and is retained but no longer read by anything; `video_core` replaced it additively so that no migration had to drop data.

Indexes, integrity and access control

- **Indexes.** `video_core` is indexed on `(project_id, created_at DESC)` for the workspace listing, and partially on `job_id` where it is non-null, which is the lookup that reconciliation performs. `conversations` is indexed on `project_id` and `messages` on `(conversation_id, created_at)`, the order a conversation is replayed in.
- **Integrity.** Every foreign key cascades on delete, so a removed account or project leaves nothing orphaned. A shared trigger function maintains `updated_at` on each relation that carries it.
- **Access control.** Row-level security is enabled on every relation. `projects`, `conversations` and `video_core` test `auth.uid() = user_id` directly. `messages` carries no owner column, so its policy tests ownership of the parent conversation through an `EXISTS` subquery — the row is reachable only if the conversation it belongs to is. This is the database half of NFR-9; the other half is the project-scoping function through which the agent's tools resolve recording identifiers.

7.3 Object Storage

Two public buckets are used, one in each Supabase project.

Because the object path begins with the content hash, re-ingesting identical bytes resolves to an object that already exists and the upload is skipped entirely. The service checks for the object before writing rather than relying on overwrite semantics, so a repeated ingest of a large recording costs nothing.

Bucket	Project	Contents and policy
`project-assets`	Application	What the browser uploads, laid out per user and project. Public read; insert restricted to authenticated accounts; delete restricted to the object's owner.
`videos`	Analysis service	The canonical copy the pipeline reads, stored at {`video_id`}/{`filename`}, together with {`video_id`}/`poster.jpg`. Public read, so a playback URL is permanent and unsigned.

7.4 Analysis Record Store

One JSON document is written per recording, named {`sanitised-filename`}__{`video_id[:8]`}`.json`. The filename carries a human-readable stem so that a directory of records can be inspected directly, and the truncated content hash so that two recordings sharing a name cannot collide. On re-ingest, any existing document for the same identifier is removed before the new one is written, so a stale record can never sit alongside a current one under a different name.

Field	Type	Contents
`video_id`	string	Content hash; the identity every other store joins on.
`video_url`	string	Public URL of the canonical copy.
`storage_path`	string	Object path within the analysis bucket.
`source_url`	string	Where the recording was fetched from.
`filename`	string	Original name, preserved for display.
`preset`, `params`	string, object	The chunking scheme and its duration bounds or weights.
`chunk_config`	string	The composite key identifying that scheme.
`duration`, `size_bytes`	number	Measured at ingest.
`poster_url`	string	The extracted poster frame.
`analyzers`	array	Which analyzers ran.
`chunks`	array	One entry per chunk; see below.
`aggregates`	object	One key per aggregator, holding its full result.
`aggregates_analyzers`	array	The analyzer set the aggregates were computed from, so a stale cache is detectable.

Each entry of `chunks` carries `id`, `start` and `end`, followed by *one key per analyzer that produced output for that chunk*, whose value is that analyzer's structured result — `default_video` holding the scene schema, `people` holding per-person records with their box locations, `ocr` holding the read texts, and so on. An analyzer that was gated out for a chunk simply contributes no key, so absence is represented naturally rather than as a null. This is precisely the shape a relational schema cannot accommodate without a migration per analyzer.

Two conventions apply throughout. Keys prefixed with an underscore — `_frames`, `_detector_labels` — are provenance retained for debugging: they record which frames a description was derived from and what the cheap detector believed, and they are never embedded, never copied into a vector payload and never returned by the API. And no local filesystem path is ever persisted; the cached file is re-derived from `video_id` on demand and re-downloaded if the cache is cold, so a record remains correct after the data directory moves or the cache is wiped.

7.5 Vector Store

Collection and point structure

A single Qdrant collection, `chunks`, holds one point per *(recording, analyzer, chunk span, chunk configuration)*. Each point carries all five named vectors, each 384-dimensional and compared by cosine distance. Making the point — rather than the vector — the unit of storage means a limit of ten returns ten chunks, the payload is stored once, and a filter is evaluated once.

Named vector	Embedded text
`combined`	The whole flattened record. The default search space.
`description`	The prose description alone.
`people`	Role, clothing and appearance, phrased as clauses.
`actions`	What each subject did, or what each object was used for.
`objects`	Object names, or the literal strings read from screen.

A vector space is omitted rather than written empty, so a chunk in which no person was seen is not a candidate in a people-scoped search at all, instead of matching against empty text.

Point identity

The point identifier is a UUID derived deterministically from

```
sha1(video_id | analyzer_id | start | end | chunk_config)
```

so that re-ingesting the same chunk upserts in place rather than duplicating. The key is built from the *time span* rather than a positional index, because chunk 12 of one chunking run is a different moment from chunk 12 of another and a positional identifier would silently rebind a vector to the wrong timeframe. Custom chunking weights are hashed into `chunk_config` for the same reason: without it, two different weightings sharing the same duration bounds would collide and the second ingest would overwrite the first.

Payload and filters

The payload carries everything required to rank, filter and cite a moment, so retrieval never joins back to the record store: the recording identifier and playback URL, the analyzer, chunk configuration and chunk identifier, the time span, the combined text and description, the setting, the people, objects, actions and tags, the speakers and their turns, the texts read from screen, the object detections, the full per-person records, and the person count.

The filter set is declared in one place and the API passes a flat dictionary through to it, so a filter added to the declaration is reachable from the API immediately; the previous hand-plumbed parameter list left six store filters silently unreachable. An unrecognised filter name raises an error carrying the nearest valid name, because a filter that is silently ignored returns plausible but wrong results, which is worse than a failure.

The same twelve fields are declared as payload indexes. The person count is indexed specifically because counting is what embeddings cannot do: a query for *"mostly empty store"* retrieved the busiest chunk in the recording, since a description listing five people reads much the same to a vector as one listing two. In embedded mode Qdrant ignores payload indexes and filtering scans instead — correct, and acceptable at this scale. The index declarations are retained so that pointing the same code at a Qdrant server gains real indexes with no change.

Deletion is scoped rather than wholesale. Removing vectors takes an optional chunk configuration and analyzer identifier, because re-running one analyzer must delete only that analyzer's

Filter	Payload field	Match	Type
video_ids	video_id	any	list[str]
chunk_ids	chunk_id	any	list[int]
analyzer_ids	extractor_id	any	list[str]
chunk_config	chunk_config	exact	str
objects	objects	any	list[str]
tags	tags	any	list[str]
speakers	speakers	any	list[str]
people	people	any	list[str]
min_people	people_count	$\geq$	int
max_people	people_count	$\leq$	int
after	start	$\geq$	float
before	end	$\leq$	float

vectors; deleting by recording alone would destroy every other analyzer's work, so adding one analyzer to an existing recording would silently discard the rest.

7.6 Cross-Store Identity

The content hash is the single join key across all four stores. It is computed while the bytes are being read, which is why the download must complete before the object's own name exists.

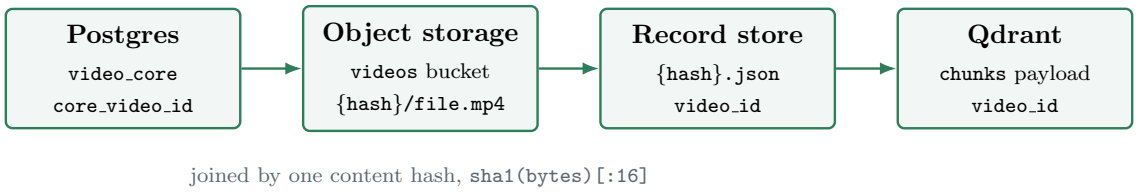

Figure 5: The content hash of the video bytes is the identity shared by every store, which is what makes ingestion idempotent (NFR-10).

Durability differs by store, and the difference is intentional:

- The **record store** is authoritative. It holds the analyzer and aggregator output, which is the expensive artifact, and everything else can be rebuilt from it.
- The **vector store** is derived. Re-indexing rebuilds it from the records at the cost of local embedding alone, with no API spend and no re-analysis.
- The **media cache** is regenerable. Deleting it costs a re-download and never data, which is why no local path is persisted anywhere.
- **Job state** is deliberately in memory and does not survive a restart. The application detects the loss when a poll returns nothing, and either recovers the recording from its persisted record or fails the row explicitly so that re-indexing becomes the route forward.

8 User Interface and Acceptance Planning

8.1 Interface Design

The interface is organised around a single principle: the operator should never be required to scrub. Every screen terminates in a moment that plays. Figure 6 shows the four principal states of the application — the project workspace, the ingestion dialog, the agent chat with its clip reel and timeline studio, and the per-recording detail view.

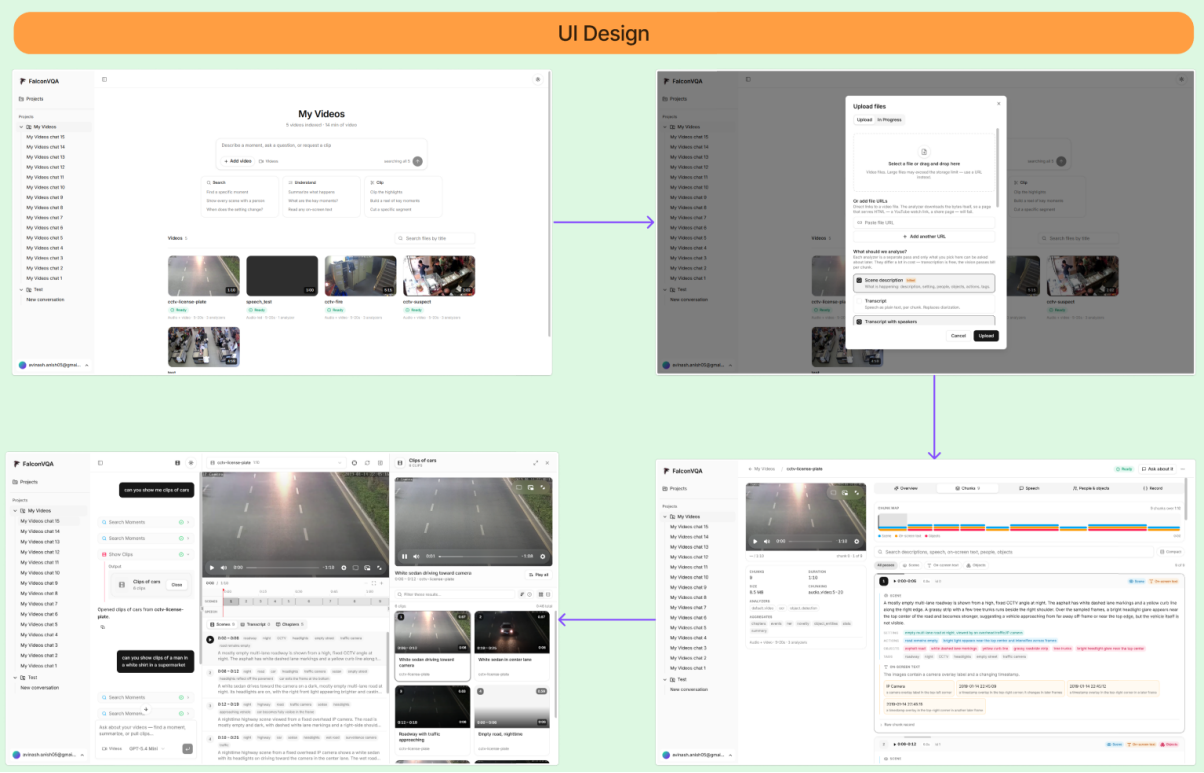

Figure 6: Operator interface. Clockwise from top left: the project workspace and video grid; the ingestion dialog, whose analyzer list is built from the analysis service's own capability endpoint; the per-recording detail view with its chunk map and tabbed record; and the agent chat with the clip reel artifact panel and the timeline studio.

Screen	Operator activity
Landing and documentation	Presents the system and hosts the full documentation: concepts, workspace guides, and the API reference.
Authentication	Registration and sign-in. Every subsequent route is guarded, and every query is restricted to the signed-in account.
Projects	Lists projects, each grouping a set of recordings. Opening one enters its workspace.
Workspace	Adds recordings by file or by link, selects the analyzers and the chunking mode, and observes each recording advance through *Uploading*, *Indexing* and *Ready*. Only ready recordings may be searched, and the interface states this rather than returning an empty result.
Agent chat	Accepts a description or a question, and a selection of recordings to consider. The assistant chooses the retrieval operations, and answers remain concise and cite timecodes. Conversations persist and may be resumed.
Clip reel panel	Opens beside the chat whenever the results are moments. A player is placed above the list of moments; selecting one plays it, and they may be played consecutively as a reel.
Timeline studio	Presents the recording's scenes, transcript and chapters against a timeline that follows the playhead, for reviewing what was indexed in the region being watched.
Recording detail	Five tabs over one recording: *Overview* for the video-level output, *Chunks* for the chunk map and per-chunk records, *Speech* for the transcript, *People & objects* for linked entities and their timelines, and *Record* for the raw stored document.

8.2 Acceptance Test Plan

Each test is executed against a live server on the datasets listed in the following subsection, and traces to the requirement it covers.

ID	Test	Passes when	Covers
AT-1	Account isolation	A second account signing in sees none of the first account's projects, recordings or conversations, and cannot reach them by identifier.	FR-1, NFR-9
AT-2	Add a recording	A file and a link both reach *Ready*, and the analyzers that ran are those selected.	FR-2
AT-3	Progress is visible	Stage and progress update during processing, and the interface remains usable throughout.	FR-3, NFR-3
AT-4	Search by description	A described event returns the correct moment within the leading results.	FR-4
AT-5	Filtered search	Restricting by time and by content removes non-matching results and retains matching ones.	FR-5
AT-6	Ask a question	The answer is correct and every claim carries a timecode reference.	FR-6, NFR-5
AT-7	Unanswerable question	The system states that the footage does not support an answer instead of producing one.	NFR-5
AT-8	Play the moment	Selecting a result begins playback at that timecode without scrubbing, and several moments from one recording play consecutively as a reel.	FR-7, NFR-4
AT-9	Video-level review	Summary, chapters, events, entities and anomalies are produced and legible in the detail view.	FR-8
AT-10	Conversational retrieval	A multi-turn conversation selects the appropriate tools without the recording being named each turn, and is intact when resumed.	FR-9
AT-11	Index inspection	The stored record for any chunk is retrievable, and the timeline follows the playhead during playback.	FR-10
AT-12	Re-add and re-examine	The same file re-added is recognised as the same recording; re-examining at another depth preserves earlier results.	FR-11, NFR-10
AT-13	API discovery	A client constructed solely from the capability endpoint issues a valid search, and a newly registered analyzer appears without a client change.	FR-12, NFR-8
AT-14	Silent, cut-free footage	Fixed-camera footage with no speech still yields multiple chunks and usable descriptions.	NFR-6

8.3 Datasets

Dataset	Content	Property under test
Supermarket CCTV	5 minutes, 1280×720, fixed camera, no speech	The principal fixture. A recording with neither cuts nor audio — precisely the case on which general-purpose tools fail. Used for chunking, scene, people, object and OCR tests.
Multi-speaker clip	60 seconds, several speakers	The only fixture in which speaker change, silence and speaker-attributed transcription all fire. Used for every audio path.
Roadway and incident clips	Short fixed-camera clips: licence plates, fire, suspected theft	Exercise on-screen text recognition, novelty detection and event extraction on distinct subject matter, and provide the moments used in clip-reel testing.
Extended set (planned)	Longer footage, multiple cameras, one shared subject	Required for cross-recording person linking, which cannot be tested on footage sharing no people.

9 Execution Plan

9.1 Schedule

Development proceeds in eight phases. Each phase is validated end to end before the next is begun, so that a defect in an earlier stage is not discovered through the behaviour of a later one.

Phase	Stage	Work
Phase 1	Chunking	Boundary detectors, weighted fusion, the three chunking modes and the duration bounds. Validated on cut-free footage before anything was built upon it.
Phase 2	Analysis	Scene, people, object, text and speech analyzers; frame sampling; detector gating; and the registry that renders them interchangeable.
Phase 3	Indexing and retrieval	Local embedding, the five named vectors, the typed filter set, response detail levels and the search endpoint.
Phase 4	Aggregation	Video-level output, dependency ordering, caching, and person and object entity linking.
Phase 5	Question answering	Routing of questions to the aggregates capable of answering them, and answer synthesis with mandatory citations.
Phase 6	Application tier	Authentication and row-level security, projects and workspace, upload and status reconciliation, the agent tool layer, clip reel and timeline studio, and the per-recording detail view.
Phase 7	Documentation	The MDX documentation site — concepts, workspace guides and the complete API reference — and the public landing pages.
Phase 8	Hardening and reporting	Acceptance tests against a live server, error handling, deployment, and this document.

9.2 Distribution of Work

The project is undertaken by two members. The division follows the pipeline: everything from a recording arriving to its being chunked and stored on one side, and everything from a chunk being analysed to a question being answered on the other.

Member	Responsibility
Avinash Anish	**Application tier and ingestion.** The entire interface — authentication, projects and workspace, upload and status, agent chat, clip reel panel, timeline studio and the recording detail view — together with the agent tool layer and its project scoping. On the pipeline side, ingestion: source handling, object storage and caching, content-addressed identity, the four boundary detectors, weighted fusion and the three chunking modes, and the background job and progress reporting behind them.
Vaivaswat Verma	**Analysis service and retrieval.** Everything downstream of chunking: the analyzers with their frame sampling and detector gating, embedding and the vector store, filters and detail levels, the video-level aggregators including person and object entity linking, and question routing and answer synthesis. Owns the HTTP API through which these are exposed.
Shared	Architecture and design decisions, acceptance testing against a live server, deployment, and documentation.